

Entregable 3. Consistencia y procesos distribuidos

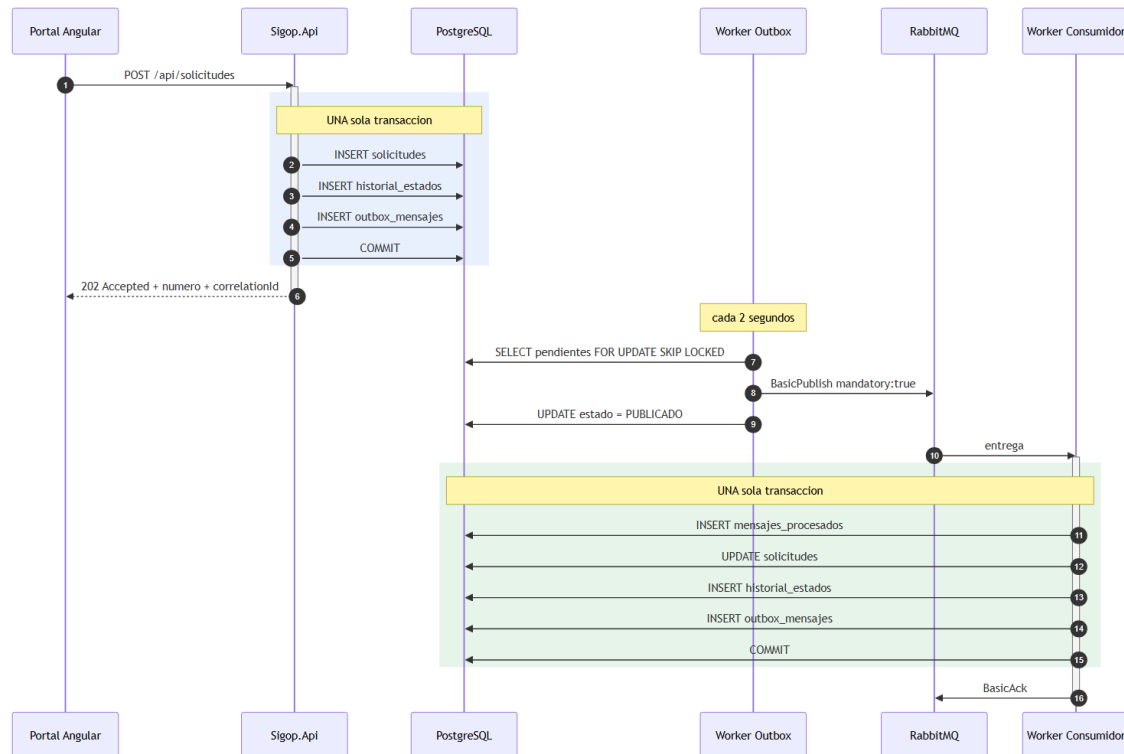
Como una operacion modifica la base de datos y publica eventos sin que exista una transaccion distribuida entre PostgreSQL y RabbitMQ, y como se orquesta una operacion que atraviesa validacion, reserva presupuestaria y confirmacion externa.

3.1 El problema

Registrar una solicitud requiere dos efectos, escribir la fila y publicar un evento, en sistemas distintos, y **no se permiten transacciones distribuidas**. Publicar primero deja eventos de algo que no existe si el COMMIT falla; persistir primero deja filas que nadie procesa si la publicacion falla. El segundo caso es justo lo que describe el enunciado: operaciones en estado incierto y reintentos manuales.

3.2 Outbox transaccional

El evento se guarda como **una fila mas de la misma transaccion**. Si la transaccion se confirma, existen los dos efectos; si se revierte, ninguno. Un proceso aparte lo publica despues.



Registro de una solicitud, publicacion del evento y consumo. Las dos transacciones estan sombreadas.

Tres detalles sostienen la garantia. El **ack va despues del COMMIT**: si el consumidor muere en medio, RabbitMQ reentrega y la idempotencia impide el doble efecto. La **marca de idempotencia se escribe en el mismo SaveChanges que el efecto**, asi que no hay ventana en la que ocurra uno sin el otro. Y **FOR UPDATE SKIP LOCKED** permite varias instancias del despachador tomando lotes distintos sin bloquearse.

El volcado al outbox ocurre antes de guardar, dentro de la misma unidad de trabajo:

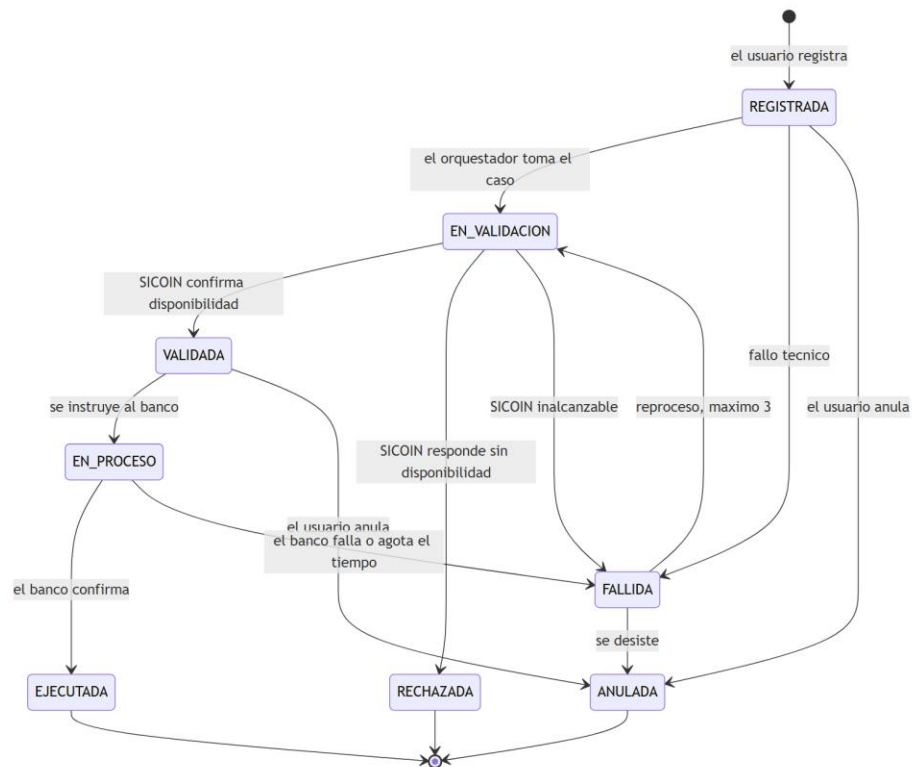
```
public async Task<int> ConfirmarAsync(Cancellation token ct = default)
{
    await VolcarEventosAlOutboxAsync(ct);
    return await contexto.SaveChangesAsync(ct);
}
```

La garantía resultante es **al menos una entrega**. Combinada con la idempotencia del consumidor, el efecto observable es de exactamente una.

3.3 Saga: validacion, reserva y confirmacion

Orquestada, no coreografiada: un solo componente conoce el flujo completo. Con tres pasos y compensaciones que dependen del estado, la claridad importa mas que el desacoplamiento.

La solicitud no tiene una tabla de saga. Su propio estado es el estado de la saga, y el historial es el registro de la ejecución. Una tabla aparte sería un segundo lugar donde la verdad puede divergir.



Maquina de estados. Las transiciones son exactamente las que declara la clase MaquinaEstados.

Cualquier transición no declarada lanza **ExcepcionReglaNegocio** con un código estable, y esa validación vive en el dominio: ni la API ni el consumidor pueden saltársela.

Paso	Participante	Exito	Fallo de negocio	Fallo tecnico
1. Validar disponibilidad	SICOIN, SOAP	VALIDADA	RECHAZADA, terminal	Reintentos y luego FALLIDA
2. Reservar presupuesto	SICOIN	EN_PROCESO	RECHAZADA	Reintentos y luego FALLIDA
3. Confirmar con el banco	Pasarela bancaria	EJECUTADA	FALLIDA, reprocesable	Reintentos y luego FALLIDA

El enunciado pide una saga con validacion, reserva presupuestaria y confirmacion externa. En el piloto el paso 2 no es una segunda llamada a SICOIN: la transicion a EN_PROCESO marca el compromiso del presupuesto y deja el punto donde esa llamada encajaria. Los pasos 1 y 3 si invocan servicios externos.

El consumidor decide por el **estado actual**, no por el tipo de evento recibido:

```
switch (solicitud.Estado)
{
    case EstadoSolicitud.Registrada:
        solicitud.IniciarValidacion(...);
        await ValidarAsync(...);
        break;
    case EstadoSolicitud.Validada:
        solicitud.IniciarEjecucion(...);
        await ConfirmarAsync(...);
        break;
}
```

Esto es lo que hace la saga inmune al desorden y a los duplicados: un mensaje que llega tarde encuentra un estado que ya no corresponde, lo registra y no hace nada.

3.4 Compensaciones

No hay ROLLBACK posible entre sistemas: una reserva presupuestaria confirmada en un sistema financiero no se deshace porque el banco falle después.

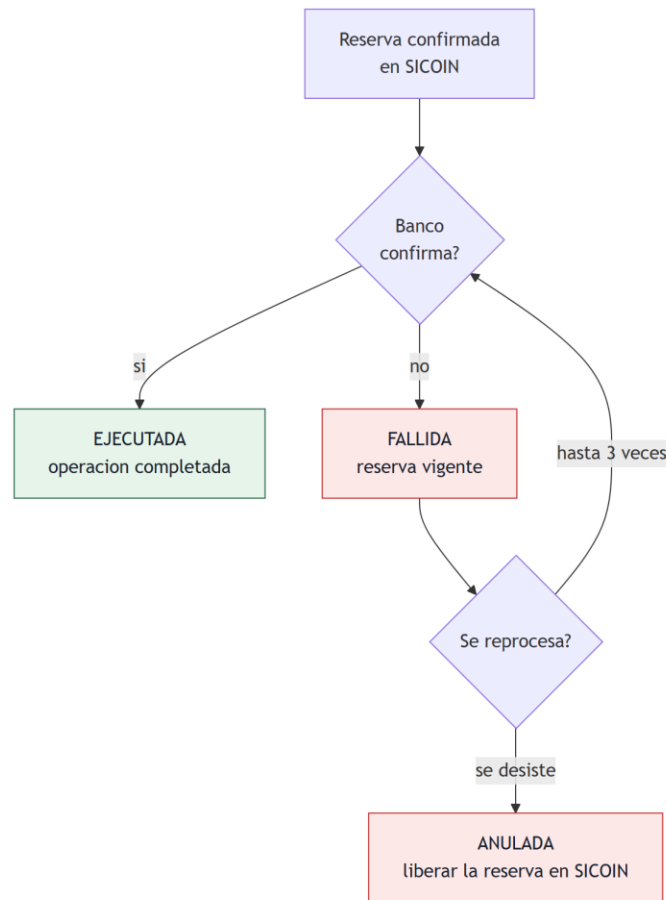


Figura 3. Que ocurre después de una reserva confirmada.

Paso confirmado	Compensacion	Cuando se dispara
Validacion	Ninguna: es una consulta sin efecto	-
Reserva presupuestaria	Liberar la reserva en SICOIN	Al anular una solicitud fallida
Confirmacion bancaria	No se compensa	Una vez acreditado se corrige por reverso contable, no automaticamente

El ultimo caso es deliberado: un reverso bancario automatico es mas peligroso que el fallo que intenta corregir. Exige intervencion humana y queda registrado como operacion nueva.

3.5 Estados intermedios

EN_VALIDACION y EN_PROCESO existen precisamente para que **no haya operaciones en estado incierto**, que es uno de los problemas del enunciado.

Una solicitud detenida en EN_PROCESO no es una incognita: significa que se instruyo al banco y no hubo respuesta aun. El historial dice cuando y con que correlacion, y el outbox si el mensaje salio. Un operador responde que paso sin abrir un depurador.

3.6 Verificacion

Los tres caminos son deterministas y reproducibles, y se comprobaron de punta a punta:

Monto	Legado	Banco	Estado final	Tiempo
Q125,000	Disponible	Confirma	EJECUTADA	Menos de 4 s
Q780,000	Disponible	Agota el tiempo	FALLIDA, reprocesable	Menos de 10 s
Q2,300,000	Sin disponibilidad	No se invoca	RECHAZADA	Menos de 4 s
Cualquiera, con SICOIN caido	Inalcanzable	No se invoca	FALLIDA tras 3 reintentos	Unos 110 s

Estado del outbox tras las pruebas: 13 mensajes, todos publicados, ninguno pendiente ni fallido, y 13 filas en mensajes_procesados, una por mensaje y sin duplicados.

El umbral del banco debe estar **por debajo** del techo del legado. Si no, la banda que termina en FALLIDA seria inalcanzable porque el legado rechazaria antes. Son dos constantes en archivos distintos y nada obliga a que se respeten entre si: es una dependencia implicita que conviene conocer.